

CS 3210 HW2

2.4

LSL X9, X0, #3 // $X9 = f * 8$: Over here, we are performing a left shift on the contents of X0, which represents f , by 3 bits. Since, shifting by 3 bits is basically equivalent to multiplying by 8, we can say that $X9 = f * 8$.

ADD X9, X6, X9 // $X9 = \&A[f]$: In the next statement, we are adding base address of array, X6, to X9. Over here, we are basically calculating the address of $A[f]$ since X9 now holds the byte offset of f index into A and adds X6 adds the base address.

LSL X10, X1, #3 // $X9 = \&A[f]$: Similar to the first statement, over here we are shifting the contents of X1 to its left by 3 bits, which effectively calculates the offset $g * 8$ in array B.

ADD X10, X7, X10 // $X10 = \&B[g]$: It basically adds the base address of array B, which is X7, to the offset $g * 8$, which we calculated in the previous step, which effectively points to $B[g]$.

LDUR X0, [X9, #0] // $f = A[f]$: This instruction basically loads a 64-bit integer from the address that is contained in X9 to X0.

ADDI X11, X9, #8: This instruction basically calculates the address of the next element in array A by adding 8 to X9 and X11 now points to $A[f+1]$

LDUR X9, [X11, #0]: Over here, we basically load the integer at $A[f+1]$ into register X9.

ADD X9, X9, X0: Over here, through this instruction we are basically adding the values of $A[f]$ and $A[f+1]$ and then stores it in X9.

STUR X9, [X10, #0]: This instruction basically stores the addition back to the address $B[g]$, which is then pointed towards X10.

From the steps, that we just performed, we can say that the corresponding C statement would be $B[g] = A[f] + A[f + 1]$, since this statement basically captures the logic of loading data from an array, performing an addition, and storing the result in another array, which is consistent with the original LEGv8 assembly instructions.

2.9

ADDI X9, X6, #8: Over here, this instruction basically adds 8 to the base address of array A and since each element is 8 bytes, it means that $X9 = \&A[1]$, which means that X9 points to the second element of A.

ADDI X10, X6, XZR: It adds X6 and 0 and then it basically copies the base address of A into X10 which means that $X10 = \&A[0]$

STUR X10, [X9, #0]: It basically stores that value in X10 into memory at the location pointed to X9, which means $A[1] = (\text{long})\&A[0]$.

LDUR X9, [X9, #0]: It basically loads the value stored into X9, which means that $X9 = A[1]$.

ADD X0, X9, X10: It basically adds the values in X9 and X10 and stores the result in X0, which can be represented as $f = A[1] + \text{long}\&A[0]$

Hence, by looking at the above statements we can say that the C equivalent for this code can be represented as follows:

```
A[1] = (long)&A[0];  
f = A[1] + (long)&A[0];
```

2.15

Let us first try and identify the instruction type. Since opcode 0x658 corresponds to ADD(register), we can say that it is a R-instruction format.

Now, we know that the R-instruction format can be represented as ADD Rd, Rn, Rm, so we can say that after plugging in the values of the register, it will look something like this: ADD X17, X15, X13

Now, let us try and write the Binary representation format over here.

Field	Bits	Value
op	[31-21]	0x658 = 11001011000
Rm	[20-16]	13 = 01101
shamt	[15-10]	0 = 000000
Rn	[9-5]	15 = 01111

Rd	[4-0]	17 = 10001
----	-------	------------

Now, let us try and combine all the fields over here:

op (11 bits): 11001011000

Rm (5 bits): 01101

shamt (6 bits): 000000

Rn (5 bits): 01111

Rd (5 bits): 10001

Full binary: 11001011000 01101 000000 01111 10001 (this is what we get)

Hence, we know that our instruction type is R-type, our assembly instruction is ADD X17, X15, X13, and our binary representation is 11001011000 01101 000000 01111 10001.

2.16

Let us first try and determine the instruction type over. Since we know that the Opcode 0x7C2 corresponds to LDUR (a load register), we can say that it uses a D-type instruction format.

Next, let us write the assembly code, where we know that the instruction format for D-type can be represented as follows: LDUR Rt, [Rn, #const], so now let us substitute the register values over here, which gives us this format: LDUR X3, [X12, #4].

Now, let us try and figure out the binary representation for this D-type format.

Field	Bits	Value
op	[31-21]	0x7C2 = 11111000010
const	[20-12]	0x4 = 000000100
Rn	[9-5]	12 = 01100
Rt	[4-0]	3 = 00011

Now if we put it all together then we get this value: 11111000010 000000100 01100 00011

Hence, our final answer is that instruction type is D-type, the assembly instruction is LDUR X3, [X12, #4], and I have a binary representation of 11111000010 000000100 01100 00011.

2.17.1

If we try and expand the register file to 128, it will mean that each of these register fields must be able to identify 128 different registers, which requires 7 integers instead of 5, which increases the instruction set to 4 times as many instructions, which means that the opcode field must grow by 2 bits.

Now, if we try and keep the total instructions at 32 bits, we know that space must be taken from another field, which in turn will shift the amount field, so it will either shrink or may be removed.

2.17.2

Over here, we can say that the register fields would increase from 5 to 7 bits, and the opcode would increase by 2 bits again in this case, and in order to keep the total instruction length the same, the immediate field would have to shrink by 6 bits.

2.17.3

Let us first 2 cases over here to answer both the sub-parts of this question

Case 1: There is a possible decrease in program size

- 1) The CPU can hold more variables in registers instead of memory, which will reduce the load and store instructions with more registers.
- 2) In this case, with more instructions, complex operations could be done in fewer instructions, which in return will reduce the total instruction count.

Case 2: There is a possible increase in program size

- 1) In this case, we know that smaller immediate fields will mean that large constants or addresses may require multiple instructions to construct.
- 2) If the instruction width increases and goes beyond 32 bits, we know that each instruction will take up more memory.
- 3) If shift or some immediate capabilities are reduced, we know that extra instructions might be needed to perform the same work.

2.21

Given C statement: $A = C[0] \ll 4$;

Given Register Assumptions: $X11 = A$, $X13 =$ base address of array C

Let us load the first element of the array C, that is $C[0]$, which is stored at the base address in $X13$, since each element is 8 bytes, so we can represent this step in the following manner:

```
LDUR X9, [X13, #0] // X9 = C[0]
```

Now, let us shift left by 4 bits, and we know that we can perform this action using the LSL instruction since it shifts the bits to its left by the specified amount, so we can represent this step in the following manner:

```
LSL X11, X9, #4 // A = C[0] << 4
```

Hence, we can say that our final minimal instruction sequence can be represented as follows:

```
LDUR X9, [X13, #0] // Load C[0] into X9
LSL X11, X9, #4 // A = C[0] << 4
```

2.22

We have been provided with the following information in this question:

$X0 = 0x0000000000101000$

Plus, we have these instructions as well:

```
CMP X0, #0
B.GE ELSE
B DONE
ELSE: ORR X1, XZR, #2
DONE:
```

Let us take a look at the first given instruction, that is, `CMP X0, #0`. So, over here, we can see that $X0$ sets the condition flag, but it does not store the result. Since, $X0 = 0x0000000000101000$, which is a positive number, we can say that the result is greater than 0, which means that the greater than or equal condition is true in this case.

In our next instruction, `B.GE ELSE`, we can say that since the result of the previous was positive, we can say that the branch to ELSE is taken, since $X0 - 0$ will return a positive result after the CPU checks it.

We skip, B.Done, since the given instruction is positive, so we move on to the next instruction, that is ORRI, X1,XZR, #2. Over here, ORRI performs a bitwise OR between a register and an immediate value. In this instruction XZR is a zero register, which means that it always contains the value 0 and #2 means that it is the immediate value, so that means this instruction basically performs $X1 = 0 \mid 2$ and we get our result of this instruction as 2, since we know that we do a bitwise between their binary conversions, that are, 0000 and 0010 gives us 0010 as the result.

So, after this line gets executed, we can know that the value X1 becomes 0x0000000000000002.

Now, DONE: just marks the end of the program, and now, finally, we can say that the value of X1 is 2 (decimal form) and 0x0000000000000002(hexadecimal form) at the end of this program.

2.23

So, over here we basically know that the B instruction uses a 26-bit signed immediate in order to determine how far the program counter can jump. Over here, the immediate value basically shifts left by 2 bits. The target address can be calculated in the following way:

Target Address = Program counter + (SignExtended(immi26)<<2)

This basically tells us that the offset ranges from -2^{27} to $(2^{27} - 4)$ bytes.

Since the value of PC has been given as 0x2000_0000, we can say that the minimum target address is $0x2000_0000 - 0x0800_0000 = 0x1800_0000$, and the maximum target address can be calculated as $0x2000_0000 + 0x0800_0000 - 4 = 0x27FF_FFFC$.

Therefore, we can say that the B instruction can branch to any address in the range 0x1800_0000 to 0x27FF_FFFC.

2.23.2

Over here, we can see that the CBZ instruction uses a 19-bit signed immediate which gets shifted by 2 bits because of the word alignment, so we can say that the branch target can be calculated as follows:

Target Address = Program Counter + (SignExtended(immi19)<<2)

This tells that the range of the target address is from -2^{20} to $(2^{20} - 4)$ bytes.

Since the given value of Program Counter is 0x2000_0000, we can say that the minimum target address is $0x2000_0000 - 0x0010_0000 = 0x1FF0_0000$ and the maximum target address is $0x2000_0000 + 0x0010_0000 - 4 = 0x200F_FFFC$.

Hence, we can say that if the branch is taken, the PC can go to any address from 0x1FF0_0000 to 0x200F_FFFC, and if it's not taken, then the execution continues at the next instruction (PC = 0x2000_0004).

2.24

Over here, the repeat instruction basically needs to perform 2 actions that is, it must check whether a register is greater than 0 and if it is, it decrements the counter and branch to a specified label.

This operation basically requires an operand, which holds the counter and a branch offset which basically indicates the target label. The CB-type (compare and branch) is the most suitable LEGBv8 format for this kind of instruction, which is already used by instructions such as CBZ and CBNZ.

So, basically in this format, the Rt field specifies the register to check and decrement the imm19 field specifies the signed word-aligned offset for the branch target and the opcode basically identifies the instructions as RPT.

Also, over here if the value in Rt is greater than 0, the instruction basically subtracts 1 from Rt and then branches it to the label and if that doesn't happen the program continues to the next instruction.

2.24.2

The shortest sequence of LEGBv8 instructions that can perform the same operations can be described as follows:

- 1) `CMP X12, #0`: This will be our first instruction since it checks if the loop counter is greater than 0 or not.
- 2) `B.LE done`: Next we will move on to this instruction, where it skips the loop body if the counter is less than or equal to 0.
- 3) `SUBI X12, X12, #1`: Next we will move on to this instruction, where we will just decrease the loop counter by 1.
- 4) `B.loop`: This will be our last instruction, where we will branch back to the loop label to repeat it.

Hence, our set of instructions will look like this:

```
CMP X12, #0
B.LE done
```

```
SUBI X12, X12, #1
B    loop
done:
```

2.29

Optimized LEGv8 code:

```
ADDI X12, X1, #800
LOOP: LDUR X11, [X1, #0]
ADD  X0, X0, X11
ADDI X1, X1, #8
CMP  X1, X12
B.LT LOOP
```

Equivalent C code:

```
int *end = MemArray + 100;
while (MemArray < end) {
    result += *MemArray;
    MemArray++;
}
```

Over here, we can basically say that the variable *i* is being used as a counter to control the number of iterations and since it does not contribute to the computation, it can be eliminated to make the loop even more efficient, so over here, instead of using a counter we can use pointer arithmetic by comparing the memory addresses directly.

Over here, each element of *MemArray* is occupying 8 bytes, so the loop can compute the address of the last element and then it continues loading and summing up all the elements until the current address reaches that point, which basically reduces the number of executed instructions per iteration by removing the separate increment and comparison for *i*.

2.36

In this question, we are assuming that the register *X11* contains the address 0x100000000 and the data stored at that address is 0x1122334455667788. Now, first let us try and understand what the given mean:

LDURB *X10*, [*X11*, #0]: Over here, this instruction basically loads 1 byte from the memory address in *X11* into the *X10* register's lower byte.

STUR X10, [X11, #8]: This instruction basically stores the full 64-bit value from the register X10 into memory starting at the address X11 + 8.

2.36.1

In this kind of system, the MSB, which is the most significant byte of the value 0x1122334455667788 can be stored in the lowest memory address. So, at address 0x10000000, we can say that the first byte is 0x11, hence LDURB loads 0x11 into X10 and STUR then stores it as a 64-bit value, which is X10 = 0x0000000000000011 and it is also the value which is stored at the address 0x10000008.

Hence, the value stored at 0x10000008 is 0x0000000000000011.

2.36.2

In this kind of system, the LSB, Least Significant byte of the value 0x1122334455667788 is stored at the lowest memory and hence we can say that at the address 0x10000000, the first byte is therefore 0x88. Thus, the LDURB basically loads 0x88 into X10 and STUR then stores it as a 64-bit value which is 0x0000000000000088, which also gets stored at the address 0x10000008.

Hence, the value stored at 0x10000008 is 0x0000000000000088.

2.37

Over here, we know that a 64-bit constant cannot be loaded into a register with a single instruction since the immediate field cannot take more than 16 bits, so in order to construct a full 64-bit value, multiple instructions such as MOVZ(move with 0) and MOVK(move with keep)

In this case, we first use the MOVZ instruction to load a 16-bit immediate to a register and basically clear all other bits. Next, we move on to the MOVK instruction, which replaces the 16 bit segment within the register while keeping its remaining bits unchanged. In the end, each instruction basically uses the LSL(Logical Shift Left) value to position its 16-bit value correctly within the 64-bit register. So, over here our goal is to create a 64-bit constant 0x1122334455667788 and basically store it in X0 and this constant can be divided into four 16 bit segments:

Bits range	Hex Value
[15:0]	0x7788
[31:16]	0x5566

[47:32]	0x3344
[63:48]	0x122

Now, we can say that our LEGv8 assembly code will somewhat look like this:

```
MOVZ X0, 0x7788
MOVK X0, 0x5566, LSL #16
MOVK X0, 0x3344, LSL #32
MOVK X0, 0x1122, LSL #48
```

After looking at the steps that I just mentioned above, we can say that the X0 will contain the 64-bit value of 0x1122334455667788, since MOVZ first initializes the register with 16-bits, next each MOVK instruction then updates the higher 16-bit actions while preserving its already written bits. Then, finally the sequence efficiently constructs a 64-bit constant within the register without requiring any memory access.